

Problem A. Turn Into a Palindrome

时间限制: 1000 ms

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

Ali 拥有一个由 n 个小写英文字母组成的字符串 s 。此外，他还有一个同样为小写英文字母的字符 c 。花费 1 枚硬币，他可以对字符串 s 执行以下操作：

- 首先，选择一个下标 i ($1 \leq i \leq n$)；
- 然后，将字符 s_i 替换为字符 c 。

Ali 希望将字符串 s 变成一个回文串^{*}，但他不想花费太多的硬币。你的任务是计算将字符串 s 变成回文串所需花费的最少硬币数量。

^{*} 如果长度为 m 的字符串 t 满足对于所有 $1 \leq i \leq m$ 均有 $t_i = t_{m-i+1}$ ，则称其为回文串。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 500$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含一个整数 n 和一个小写英文字母 c ($1 \leq n \leq 100$)，分别表示字符串 s 的长度和给定的字符 c 。

第二行包含一个由 n 个小写英文字母组成的字符串 s 。

输出格式

对于每组测试数据，输出一个整数，表示 Ali 将字符串变成回文串所需花费的最少硬币数量。

样例

标准输入流	标准输出流
5	1
4 b	0
abca	2
3 p	2
xyx	8
5 e	
abcbb	
8 d	
adbccbad	
10 c	
codeforces	

提示

在第一组测试数据中，花费 1 枚硬币将 s_3 替换为 **b**。替换后字符串变为 **abba**，它已经是一个回文串。可以证明 1 是所需的最少硬币数量。

在第二组测试数据中，字符串 s 本身已经是一个回文串。

在第三组测试数据中，只需花费 2 枚硬币将 s_1 和 s_5 替换为 **e**。替换后字符串变为 **ebcbe**，它是一个回文串。

在第四组测试数据中，花费 2 枚硬币将 s_1 和 s_7 替换为 **d** 即可使字符串变为回文串。

Problem B. Fashionable Array

时间限制: 1000 ms

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

一个数组的**众数**是指在该数组中出现次数最多的数字。如果有多个数字出现的次数相同且最多，则众数为这些数字中的最大值。例如，数组 $[1, 1, 2]$ 的众数是 1，数组 $[3, 4]$ 的众数是 4。

给定一个由 n 个整数组成的数组 a 。你可以按任意顺序重新排列数组 a 中的数字。你的任务是重排数组 a 中的元素，使得该数组所有前缀的众数之和最大。

例如，数组 $[2, 3, 2]$ 可以重排为 $[3, 2, 2]$ 。此时所有前缀的众数之和计算如下：

- 长度为 1 的前缀是 $[3]$ ，该前缀的众数是 3。
- 长度为 2 的前缀是 $[3, 2]$ ，该前缀的众数是 3。
- 长度为 3 的前缀是 $[3, 2, 2]$ ，该前缀的众数是 2。

因此，众数之和为 $3 + 3 + 2 = 8$ 。可以证明，对于该数组 a 的这种排列，答案达到了最大值。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 500$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含一个整数 n ($1 \leq n \leq 100$)，表示数组的长度。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 100$)，表示数组中的元素。

输出格式

对于每组测试数据，输出一个重排后的新数组，使其所有前缀的众数之和达到最大。如果存在多个最优解，输出其中任意一个即可。

样例

标准输入流	标准输出流
7	3 2 2
3	4 4 3 2 1 1
2 3 2	4 1 3 2 2
6	2 1 1 1
4 4 2 1 3 1	7 1 2 3 4 5 6
5	4 3 2 1 3 3 1 2
1 3 2 4 2	4 4 3 3 2 1 1 3
4	
1 1 1 2	
7	
1 2 3 4 5 6 7	
8	
1 1 4 2 3 3 3 2	
8	
4 3 3 3 2 1 4 1	

提示

第一组测试数据已在题目描述中给出。

在第二组测试数据中，一种可行的排列是 $[4, 4, 3, 2, 1, 1]$ 。此时每个前缀的众数均为 4。

在第三组测试数据中，排列 $[4, 1, 3, 2, 2]$ 是一个有效答案。各前缀的众数如下：

- 长度为 1 的前缀是 $[4]$ ，该前缀的众数是 4。
- 长度为 2 的前缀是 $[4, 1]$ ，该前缀的众数是 4。
- 长度为 3 的前缀是 $[4, 1, 3]$ ，该前缀的众数是 4。
- 长度为 4 的前缀是 $[4, 1, 3, 2]$ ，该前缀的众数是 4。
- 长度为 5 的前缀是 $[4, 1, 3, 2, 2]$ ，该前缀的众数是 2。

此时众数之和为 $4 + 4 + 4 + 4 + 2 = 18$ 。可以证明，该排列使众数之和最大。

Problem C. GCD Treasury

时间限制: 2000 ms

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

贪婪的海盗 Dimash 发现了一座宝库。宝库中包含 n 堆硬币，编号从 1 到 n 。第 i 堆恰好包含 a_i 枚硬币。海盗 Dimash 拥有一个数字 x ，他可以使用这个数字从宝库中窃取硬币。偷窃过程如下：

- 首先，他选择一个下标 i ($1 \leq i \leq n$)，满足 $a_i > 0$ 且 $\gcd^*(a_i, x) \neq 1$ 。如果不存在这样的下标，海盗停止偷窃。
- 设 $g = \gcd(a_i, x)$ 。海盗从第 i 堆中偷走恰好 g 枚硬币，此后 a_i 减少 g 。
- 最后，他将 x 的值更新为 g ，然后继续偷窃硬币。

你的任务是帮助海盗偷取尽可能多的硬币。求出他最多能从宝库中偷取的硬币总数。

* $\gcd(a_i, x)$ 表示整数 a_i 和 x 的最大公约数 (GCD)。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 10^4$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含两个整数 n 和 x ($1 \leq n, x \leq 3 \cdot 10^5$)，分别表示宝库中硬币堆的数量以及海盗所持有的数字。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 3 \cdot 10^5$)，表示各堆中的硬币数量。

保证所有测试数据中 n 的总和不超过 $3 \cdot 10^5$ 。

输出格式

对于每组测试数据，输出一个整数，表示海盗最多能够偷取的硬币数量。

样例

标准输入流	标准输出流
5	0
3 1	6
2 3 5	8
3 4	9
2 3 4	20
4 2	
2 2 2 2	
6 6	
2 3 2 3 2 3	
7 6	
9 9 4 4 4 4 4	

提示

在第一组测试数据中，无法偷取任何硬币。

在第二组测试数据中，海盗的偷窃过程如下：

- 海盗选择第 3 堆。他偷走 4 枚硬币，此时宝库状态变为 $[2, 3, 0]$ ， x 变为 4。
- 海盗选择第 1 堆。他偷走 2 枚硬币，此时宝库状态变为 $[0, 3, 0]$ ， x 变为 2。
- 此时已没有满足条件的下标，海盗停止操作。海盗总共偷取了 $4 + 2 = 6$ 枚硬币。

Problem D. Backrooms Hill

时间限制: 2000 ms

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

在来自城市 GR 的人们中，流传着一个关于通后台的山丘 (hill) 的传说。对于一个由 m 个整数组成的数组 b ，如果存在一个下标 k ($1 \leq k \leq m$) 满足以下条件，则称数组 b 为一座山丘 (hill)：

- 前 k 个元素严格单调递增。形式化地，对于所有 $1 \leq i < k$ ，均有 $b_i < b_{i+1}$ 。
- 后 $m - k + 1$ 个元素严格单调递减。形式化地，对于所有 $k \leq i < m$ ，均有 $b_i > b_{i+1}$ 。

给定一个由 n 个互不相同的整数组成的数组 a 。你可以执行以下操作任意次：

- 选择任意下标 i ($1 \leq i \leq n - 2$)；
- 交换数组中的 a_i 和 a_{i+2} 。

你的任务是判断是否可以通过执行该操作任意次，将数组 a 变成一座山丘。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 10^4$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含一个整数 n ($1 \leq n \leq 2 \cdot 10^5$)，表示数组 a 的长度。

第二行包含 n 个互不相同的整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$)。

保证所有测试数据中 n 的总和不超过 $2 \cdot 10^5$ 。

输出格式

对于每组测试数据，如果可以将数组 a 变成一座山丘，输出 "YES"；否则，输出 "NO"。

你可以输出答案的任意大小写形式（例如，字符串 "yEs"、"yes"、"Yes" 和 "YES" 均被视为肯定回答）。

样例

标准输入流	标准输出流
7	NO
3	YES
2 1 3	YES
4	NO
1 2 3 4	NO
6	YES
5 3 4 6 2 1	NO
6	
1 5 2 3 6 4	
7	
1 2 3 4 7 5 6	
5	
5 4 1 3 2	
6	
4 1 3 2 6 5	

提示

在第一组测试数据中，无法将数组 a 变成山丘。

在第二组测试数据中，数组 a 本身已经是一座山丘。

在第三组测试数据中，可以按如下方式执行操作：

- 交换 a_1 和 a_3 。操作后，数组 a 变为 $[4, 3, 5, 6, 2, 1]$ 。
- 交换 a_2 和 a_4 。操作后，数组 a 变为 $[4, 6, 5, 3, 2, 1]$ 。

数组 $[4, 6, 5, 3, 2, 1]$ 是一座山丘。

Problem E. Clean Substrings

时间限制: 2000 ms

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

如果一个长度为 m 的 01 字符串 t 满足对于所有 $1 \leq i < m$ 均有 $t_i = t_{i+1}$ ，则称其为**整洁的** (clean)。

该国的总工程师又有了新发现。他发明了一台智能机器人，花费 1 枚硬币，机器人可以对任意 01 字符串 t 执行以下操作：

- 选择字符串 t 的任意一个**整洁子串** $*$ 。
- 将该子串中的所有字符翻转（即将 0 变为 1，将 1 变为 0）。

定义字符串 t 的**美观度** (beauty) 为将其变为整洁字符串所需的最少硬币数量。定义字符串 t 的**权值** (power) 为其所有子串的美观度之和。

现给定一个长度为 n 的 01 字符串 s 。工程师的竞争对手将对该字符串进行恰好 q 次修改。每次修改由一个整数 i 给出，表示将字符 s_i 翻转。你的任务是求出初始时以及每次修改后字符串 s 的权值。

* 如果字符串 a 可以通过从字符串 b 的开头删除若干个（可能为零个或全部）字符，并从末尾删除若干个（可能为零个或全部）字符得到，则称 a 是 b 的子串。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 10^4$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含两个整数 n 和 q ($1 \leq n, q \leq 2 \cdot 10^5$)，分别表示 01 字符串的长度和修改操作的次数。

第二行包含一个由 n 个字符组成的 01 字符串 s 。

接下来的 q 行，每行包含一个整数 i ($1 \leq i \leq n$)，描述一次修改操作的位置。

保证所有测试数据中 n 的总和与 q 的总和均不超过 $2 \cdot 10^5$ 。

输出格式

对于每组测试数据，输出一行 $q + 1$ 个由空格隔开的整数，分别表示修改前以及每次修改后字符串的权值。

样例

标准输入流	标准输出流
4	2 2
3 1	7 5 5
110	36 23 22 25 26
2	61 66 41 35 59 60
4 2	
1010	
1	
2	
8 4	
10101110	
3	
5	
2	
3	
10 5	
1100101110	
1	
6	
10	
7	
3	

提示

以第一组测试数据为例：

修改前，字符串 s 为 110，其权值为 2：

- 子串 $s[1,1]$ 的美观度为 0。
- 子串 $s[2,2]$ 的美观度为 0。
- 子串 $s[3,3]$ 的美观度为 0。
- 子串 $s[1,2]$ 的美观度为 0。
- 子串 $s[2,3]$ 的美观度为 1。
- 子串 $s[1,3]$ 的美观度为 1。

修改后（将 s_2 翻转），字符串 s 变为 100，此时权值仍为 2：

- 子串 $s[1,1]$ 的美观度为 0。
- 子串 $s[2,2]$ 的美观度为 0。
- 子串 $s[3,3]$ 的美观度为 0。
- 子串 $s[1,2]$ 的美观度为 1。

- 子串 $s[2, 3]$ 的美观度为 0。
- 子串 $s[1, 3]$ 的美观度为 1。

Problem F1. XOR Transformations (Easy Version)

时间限制: 4000 ms

内存限制: 512 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

这是问题的简单版本。两个版本之间的区别在于本版本中 n 和 q 的数据范围较小。只有在解决了该问题的所有版本后，你才能进行 **hack**。

对于一个由 m 个整数组成的数组 b ，定义一次变换（transformation）如下：

- 记录下所有满足 $1 \leq i < j \leq m$ 的 $b_i \oplus b_j$ 的值，其中 $\oplus$ 表示按位异或运算。
- 取出所记录的数值中最小的 m 个值。
- 用取出的这 m 个值替换原数组中的元素。

例如，考虑对数组 $[6, 7, 8, 9, 15]$ 进行一次变换。我们记录下的数值为 $1, 1, 6, 7, 8, 9, 14, 14, 15, 15$ 。变换后，数组变为 $[1, 1, 6, 7, 8]$ ——即最小的 5 个元素。

现给定一个由 n 个非负整数组成的数组 a 。令 $\max(a)$ 表示数组 a 中的最大元素， $\min(a)$ 表示最小元素。你的任务是回答 q 次询问，每次询问给定一个整数 x 。对于每次询问，求出对数组执行 x 次变换后的 $\max(a) - \min(a)$ 。请注意，询问之间是相互独立的，即在每次询问前，数组 a 都会恢复为其初始状态。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 100$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含两个整数 n 和 q ($5 \leq n \leq 2000, 1 \leq q \leq 2000$)，分别表示数组的大小和询问的次数。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$)。

接下来的 q 行，每行包含一个整数 x ($0 \leq x < 2^{30}$)。

保证所有测试数据中 n 的总和与 q 的总和均不超过 2000。

输出格式

对于每组测试数据，输出 q 个整数，分别表示各次询问的答案。

样例

标准输入流	标准输出流
4	1
5 1	9
0 0 1 1 1	7
1	12
5 2	31
6 7 8 9 15	7
0	
1	
8 2	
102 92 15 19 25 54 62 36	
2	
1	
10 1	
56 73 81 23 17 92 50 34 67 78	
2	

提示

在第一组测试数据中，执行一次变换后，数组变为 $[0, 0, 0, 0, 1]$ 。此时 $\max(a) - \min(a) = 1 - 0 = 1$ 。

在第二组测试数据中，初始数组为 $[6, 7, 8, 9, 15]$ 。最初 $\max(a) - \min(a) = 15 - 6 = 9$ 。在执行一次变换后，数组变为 $[1, 1, 6, 7, 8]$ ，此时 $\max(a) - \min(a) = 8 - 1 = 7$ 。

Problem F2. XOR Transformations (Hard Version)

时间限制: 4000 ms

内存限制: 512 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

这是问题的困难版本。两个版本之间的区别在于本版本中 n 和 q 的数据范围更大。只有在解决了该问题的所有版本后，你才能进行 **hack**。

对于一个由 m 个整数组成的数组 b ，定义一次变换（transformation）如下：

- 记录下所有满足 $1 \leq i < j \leq m$ 的 $b_i \oplus b_j$ 的值，其中 $\oplus$ 表示按位异或运算。
- 取出所记录的数值中最小的 m 个值。
- 用取出的这 m 个值替换原数组中的元素。

例如，考虑对数组 $[6, 7, 8, 9, 15]$ 进行一次变换。我们记录下的数值为 $1, 1, 6, 7, 8, 9, 14, 14, 15, 15$ 。变换后，数组变为 $[1, 1, 6, 7, 8]$ ——即最小的 5 个元素。

现给定一个由 n 个非负整数组成的数组 a 。令 $\max(a)$ 表示数组 a 中的最大元素， $\min(a)$ 表示最小元素。你的任务是回答 q 次询问，每次询问给定一个整数 x 。对于每次询问，求出对数组执行 x 次变换后的 $\max(a) - \min(a)$ 。请注意，询问之间是相互独立的，即在每次询问前，数组 a 都会恢复为其初始状态。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 10^4$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含两个整数 n 和 q ($5 \leq n \leq 10^5, 1 \leq q \leq 10^5$)，分别表示数组的大小和询问的次数。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$)。

接下来的 q 行，每行包含一个整数 x ($0 \leq x < 2^{30}$)。

保证所有测试数据中 n 的总和与 q 的总和均不超过 10^5 。

输出格式

对于每组测试数据，输出 q 个整数，分别表示各次询问的答案。

样例

标准输入流	标准输出流
4	1
5 1	9
0 0 1 1 1	7
1	12
5 2	31
6 7 8 9 15	7
0	
1	
8 2	
102 92 15 19 25 54 62 36	
2	
1	
10 1	
56 73 81 23 17 92 50 34 67 78	
2	

提示

在第一组测试数据中，执行一次变换后，数组变为 $[0, 0, 0, 0, 1]$ 。此时 $\max(a) - \min(a) = 1 - 0 = 1$ 。

在第二组测试数据中，初始数组为 $[6, 7, 8, 9, 15]$ 。最初 $\max(a) - \min(a) = 15 - 6 = 9$ 。在执行一次变换后，数组变为 $[1, 1, 6, 7, 8]$ ，此时 $\max(a) - \min(a) = 8 - 1 = 7$ 。

Problem G. New LRT

时间限制: 2000 ms

内存限制: 512 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

在这座著名的城市中开通了一条新的轻轨（LRT）。这列列车沿着一条直线运行。

给定两个整数 n 和 m ，以及一个由 m 个数组成的数组 c 。你当前位于位置 0，需要到达位置 n 。如果你当前位于位置 i ，你可以按如下方式乘坐列车：

- 选择一个正整数 x ，使得 $m \& x = x$ ，其中 $\&$ 表示按位与运算。
- 从位置 i 移动到位置 $i + x$ ，并支付 c_x 枚硬币。

一次出行的**花费**（cost）是从位置 0 到达位置 n 所需支付的硬币总数。如果两次出行的移动顺序不同或某次移动的跨度不同，则视为不同的出行方式。你的任务是计算所有可能出行方式的花费之和。由于答案可能很大，请将其对 $10^9 + 7$ 取模后输出。

输入格式

每个测试点包含多组测试数据。第一行包含一个整数 t ($1 \leq t \leq 10^4$)，表示测试数据的组数。随后是每组测试数据的详细描述。

每组测试数据的第一行包含两个整数 n 和 m ($1 \leq n, m < 2^{20}$)。

第二行包含 m 个整数 $c_1, c_2, \dots, c_m$ ($1 \leq c_i \leq 10^9$)。

保证所有测试数据中 n 的总和与 m 的总和均不超过 2^{20} 。

输出格式

对于每组测试数据，输出一个整数，表示所有可能出行方式的花费之和对 $10^9 + 7$ 取模后的结果。

样例

标准输入流	标准输出流
4	15
3 3	16
2 2 1	271
4 5	999997081
3 1 2 4 5	
5 3	
9 1 5	
8 3	
1000000000 1000000000 1000000000	

提示

在第一组测试数据中，共有 4 种到达位置 n 的方式：

- $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ ：此次出行的花费为 $c_1 + c_1 + c_1 = 6$ 。
- $0 \rightarrow 1 \rightarrow 3$ ：此次出行的花费为 $c_1 + c_2 = 4$ 。
- $0 \rightarrow 2 \rightarrow 3$ ：此次出行的花费为 $c_2 + c_1 = 4$ 。
- $0 \rightarrow 3$ ：此次出行的花费为 $c_3 = 1$ 。

因此，答案为 $6 + 4 + 4 + 1 = 15$ 。